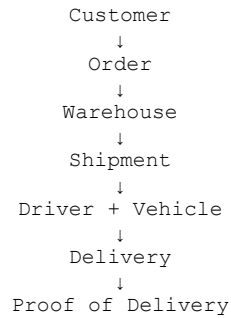


FleetFlow

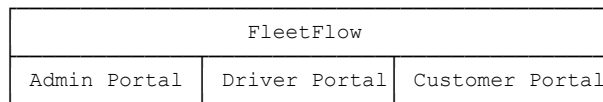
FleetFlow — Logistics & Delivery Management Platform

Core idea

A company uses FleetFlow to manage the complete lifecycle of deliveries:



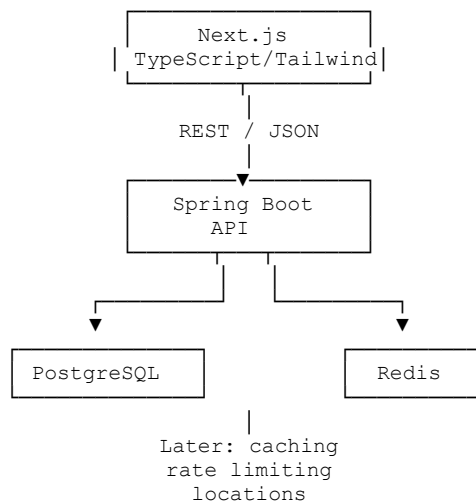
We'll eventually have three major interfaces:

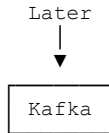


But **not yet**.

Phase 0 — Architecture

Our initial architecture:





Important: this starts as a **modular monolith**.

We're not creating:

```
user-service
order-service
shipment-service
driver-service
notification-service
payment-service
```

on day one.

That's unnecessary complexity.

Phase 1 — What we're building first

Our first version will contain only:

Authentication

- Register
- Login
- Logout
- JWT access token
 - Refresh token
- Role-based authorization

Users

- Admin
- Dispatcher
 - Driver
- Customer

Customers

- Create customer
- Update customer
- View customer
- Customer addresses

Vehicles

- Register vehicle
- Vehicle status
- Vehicle type
- Vehicle capacity

Drivers

- Driver profile
- Driver availability
- Driver ↔ vehicle assignment

Orders

- Create order
- Order status
- Pickup address
- Delivery address
- Package information

Shipments

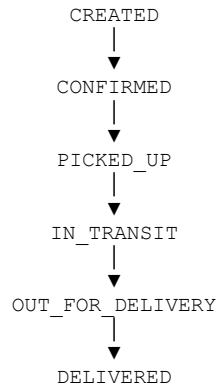
- Create shipment from order
 - Assign driver
 - Assign vehicle
- Shipment lifecycle

That's enough for our **MVP**.

Phase 2 — Shipment lifecycle

This is one of the most important parts.

We'll define a proper state machine.



Failure paths:

CONFIRMED → CANCELLED
IN_TRANSIT → DELIVERY_FAILED
OUT_FOR_DELIVERY → DELIVERY_FAILED

We **won't** allow arbitrary status changes.

For example:

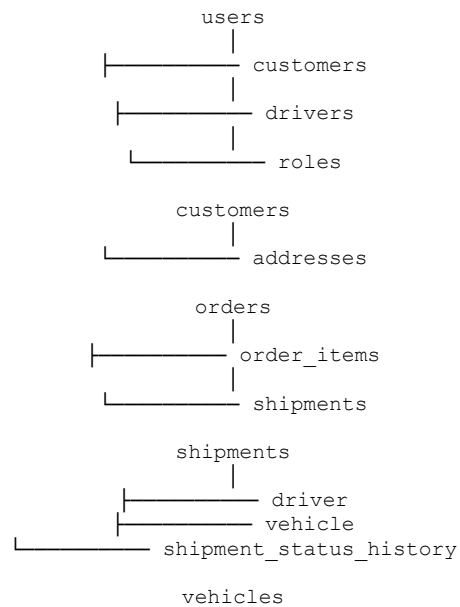
CREATED → DELIVERED

should be rejected.

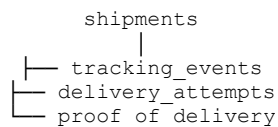
That gives us a legitimate business rule that belongs in the backend.

Phase 3 — Database

Our initial PostgreSQL model:



Eventually:



Phase 4 — Backend structure

We'll use feature-based packaging:

```
backend/
└─ src/main/java/com/fleetflow/
    │
    │ └─ auth/
    │   │
    │   │ └─ controller/
    │   │   │
    │   │   │ └─ service/
    │   │   │   │
    │   │   │   │ └─ dto/
    │   │   │   │   │
    │   │   │   │   └─ security/
    │   │   │   │       │
    │   │   │   │       └─ user/
    │   │   │   │           │
    │   │   │   │           └─ customer/
    │   │   │   │               │
    │   │   │   │               └─ driver/
    │   │   │   │                   │
    │   │   │   │                   └─ vehicle/
    │   │   │   │                       │
    │   │   │   │                       └─ order/
    │   │   │   │                           │
    │   │   │   │                           └─ shipment/
    │   │   │   │                               │
    │   │   │   │                               └─ tracking/
    │   │   │   │                                   │
    │   │   │   │                                   └─ notification/
    │   │   │   │                                       │
    │   │   │   │                                       └─ analytics/
    │   │   │   │                                           │
    │   │   │   │                                           └─ common/
    │   │   │   │                                               │
    │   │   │   │                                               └─ exception/
    │   │   │   │                                                   │
    │   │   │   │                                                   └─ response/
    │   │   │   │                                                       │
    │   │   │   │                                                       └─ validation/
    │   │   │   │                                                           │
    │   │   │   │                                                           └─ audit/
    │   │   │   │                                                               │
    │   │   │   │                                                               └─ config/
```

Every feature gets its own:

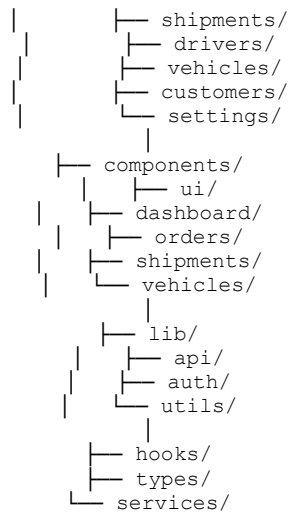
```
Controller
Service
Repository
Entity
DTO
Mapper
```

where appropriate.

Phase 5 — Frontend

Next.js:

```
frontend/
└─ src/
    │
    │ └─ app/
    │   │
    │   │ └─ login/
    │   │   │
    │   │   │ └─ register/
    │   │   │   │
    │   │   │   │ └─ dashboard/
    │   │   │   │   │
    │   │   │   │   └─ overview/
    │   │   │   │       │
    │   │   │   │       └─ orders/
```



We'll use:

- Next.js
- TypeScript
- Tailwind
- shadcn/ui
- React Hook Form
 - Zod

Phase 6 — API conventions

Every endpoint will follow:

`/api/v1/...`

For example:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
POST /api/v1/auth/refresh

GET /api/v1/customers
POST /api/v1/customers
GET /api/v1/customers/{id}
PATCH /api/v1/customers/{id}

GET /api/v1/drivers
POST /api/v1/drivers

GET /api/v1/vehicles
POST /api/v1/vehicles

POST /api/v1/orders
GET /api/v1/orders
GET /api/v1/orders/{id}

POST /api/v1/shipments
GET /api/v1/shipments
```

```
GET    /api/v1/shipments/{id}

PATCH /api/v1/shipments/{id}/status
PATCH /api/v1/shipments/{id}/assign-driver
```

We'll also standardize responses and errors.

Example:

```
{
  "success": true,
  "data": {},
  "message": "Shipment created successfully",
  "timestamp": "2026-08-12T19:30:00Z"
}
```

And errors:

```
{
  "success": false,
  "error": {
    "code": "SHIPMENT_NOT_FOUND",
    "message": "Shipment with id 123 was not found"
  },
  "timestamp": "2026-08-12T19:30:00Z"
}
```

Phase 7 — Real engineering features

Once the MVP works, we'll add the things that make FleetFlow genuinely impressive.

Redis

Use it for:

```
Popular shipment queries
Driver availability
Rate limiting
Temporary location data
```

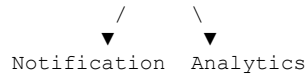
Kafka

Events such as:

```
OrderCreated
ShipmentCreated
ShipmentStatusChanged
DriverAssigned
ShipmentDelivered
DeliveryFailed
```

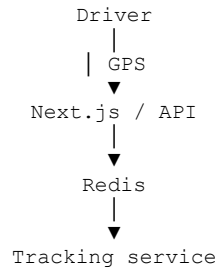
For example:

```
Shipment Status Changed
  ↓
  Kafka
```



Tracking

Eventually:



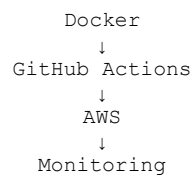
Analytics

Dashboard:

Total Shipments
Delivered
In Transit
Failed
Average Delivery Time
Driver Utilization
Vehicle Utilization

Phase 8 — Production

Then:



We'll add:

- Docker Compose
 - CI pipeline
- integration tests
- Testcontainers
- Spring Actuator
- structured logging
 - metrics
 - health checks
- database migrations
- API documentation
- security hardening

Our development rule

We're going to build **vertically**, not horizontally.

Bad approach:

```
Build every entity
↓
Build every repository
↓
Build every controller
↓
Build frontend
↓
Try to connect everything
```

Good approach:

```
Authentication
├── DB
│   ↓
│   Entity
│   ↓
│   Repository
│   ↓
│   Service
│   ↓
│   Controller
│   ↓
│   Security
│   ↓
│   Test
│   ↓
│   Next.js
│   ↓
└── End-to-end test
```

Then:

```
Customer
├── DB → API → Frontend → Test
```

Then:

```
Vehicle
├── DB → API → Frontend → Test
```

And so on.

This exposes integration problems early instead of leaving you with a giant pile of unfinished code.

🚀 Step 1 — Create the project

We're starting with the backend.

Use:

Java 24

Spring Boot

Maven

PostgreSQL

Initial dependencies:

```
Spring Web
Spring Data JPA
Spring Security
Validation
PostgreSQL Driver
Flyway
Lombok
Spring Boot Actuator
```

Do **not** add Kafka, Redis, AWS SDK, Elasticsearch, etc. yet.

Our first backend target is:

```
Spring Boot
|
├─ PostgreSQL
|
├─ Flyway
|
├─ JPA
|
└─ Health endpoint
```

Then we'll create the first real domain:

User

and implement:

```
POST /api/v1/auth/register
POST /api/v1/auth/login
```

Once authentication works end-to-end, we'll move into **Customer → Vehicle → Driver → Order → Shipment**.

And I'll be strict about the architecture as we go; if you start taking shortcuts that would make the project look like another college CRUD app, I'll call it out.